Clustering: Techniques, Evaluation, and Applications

1 Introduction

Clustering is an unsupervised machine learning technique used to group similar data points without predefined labels. This document covers clustering methods such as K-Means, centroid initialization, color similarity in image segmentation, Elkan's Accelerated K-Means, and evaluation techniques including Silhouette Score, Adjusted Rand Index (ARI), and Cross-Validation in clustering.

2 K-Means Clustering

K-Means clustering partitions N data points into K clusters by minimizing within-cluster variance. The algorithm proceeds as follows:

1. **Initialization:** Randomly select K data points as initial centroids.
2. **Assignment:** Assign each data point to the closest centroid based on Euclidean distance.
3. **Update:** Recompute each centroid as the mean of all points assigned to it.
4. **Convergence:** Repeat until centroids stabilize or the maximum number of iterations is reached.

2.1 Centroid Initialization with K-Means++

A robust method to initialize centroids, K-Means++ was proposed by Arthur and Vassilvitskii in 2006. It spreads out initial centroids as follows:

1. Select one initial centroid c_1 uniformly at random from the dataset.
2. For each subsequent centroid c_i, choose a point x_i with probability proportional to $D(x_i)^2$, where $D(x_i)$ is the distance between x_i and the closest chosen centroid.
3. Repeat until K centroids are chosen, which helps to minimize the clustering objective function.

2.2 Numerical Example for K-Means++ Initialization

Consider a 2D dataset with points at coordinates $(1, 1)$, $(2, 2)$, $(8, 8)$, and $(9, 8)$. Suppose $(1, 1)$ is chosen as the first centroid. The squared distances are:

$$D(2,2)^2 = (1-2)^2 + (1-2)^2 = 2$$
$$D(8,8)^2 = (1-8)^2 + (1-8)^2 = 98$$
$$D(9,8)^2 = (1-9)^2 + (1-8)^2 = 113$$

Total squared distance $= 2 + 98 + 113 = 213$. Probabilities: $2/213 \approx 0.009$, $98/213 \approx 0.460$, $113/213 \approx 0.531$. Thus, points further away like $(8, 8)$ and $(9, 8)$ have higher probability of selection, promoting well-spaced initial centroids.

3 Color Similarity in Image Segmentation with K-Means

K-Means can be applied to image segmentation by grouping pixels into clusters based on *color similarity*.

3.1 Color Similarity

Each pixel's color is represented in a color space like RGB. The similarity between two pixels $p_1 = (R_1, G_1, B_1)$ and $p_2 = (R_2, G_2, B_2)$ is quantified by their Euclidean distance:

$$d(p_1, p_2) = \sqrt{(R_1 - R_2)^2 + (G_1 - G_2)^2 + (B_1 - B_2)^2}$$

3.2 Application to Image Segmentation

By clustering pixels based on color similarity, K-Means groups pixels into color-based clusters, effectively segmenting the image. Each centroid corresponds to a color, with clusters representing different image regions.

4 Elkan's Accelerated K-Means

Elkan's method accelerates K-Means by using triangle inequality to avoid unnecessary distance calculations between points and centroids. Instead of recalculating all distances in every assignment step, the algorithm maintains and updates two data structures for each point P:

- **Upper bound** $u(P)$: An upper bound on the distance from P to its currently assigned centroid.
- **Lower bounds** $l(P, C_j)$: A lower bound on the distance from P to each other centroid C_j.

The key insight is that if the upper bound $u(P)$ is *less than* the lower bound $l(P, C_j)$ for some centroid C_j, then P cannot be closer to C_j than to its current centroid, and the distance $d(P, C_j)$ does not need to be computed.

4.1 Mathematical Foundation

The method relies on the triangle inequality: for any points P, C_1, and C_2:

$$d(P, C_2) \geq |d(C_1, C_2) - d(P, C_1)|$$

However, in practice, Elkan's algorithm maintains more sophisticated bounds:

- After centroid movement, update upper bounds: $u_{new}(P) = u_{old}(P) + d(C_{old}, C_{new})$
- Update lower bounds: $l_{new}(P, C_j) = \max(0, l_{old}(P, C_j) - d(C_j^{old}, C_j^{new}))$
- If $u(P) \leq l(P, C_j)$ for any centroid $C_j \neq C_{current}$, then P cannot be closer to C_j than to its current centroid

4.2 Numerical Example for Elkan's Method

Consider two centroids $C_1 = (1, 1)$ and $C_2 = (8, 8)$ and a point $P = (2, 2)$. After centroid movement, suppose we have:

$$\begin{aligned} u(P) &= 2.5 \quad \text{(upper bound to current centroid } C_1\text{)} \\ l(P, C_2) &= 7.0 \quad \text{(lower bound to centroid } C_2\text{)} \end{aligned}$$

Since $u(P) = 2.5 < l(P, C_2) = 7.0$, we can guarantee that P is closer to C_1 than C_2 without computing the actual distance $d(P, C_2)$.

5 Silhouette Score for Clustering Evaluation

The Silhouette Score assesses both cohesion and separation, ranging from -1 to 1.

5.1 Mathematical Definition

For a data point i, the Silhouette Score is:

$$S(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

where:

- $a(i)$ is the average distance between i and other points in the same cluster.
- $b(i)$ is the minimum average distance between i and points in the nearest different cluster.

5.2 Cohesion Term $a(i)$

$a(i)$ is computed as:

$$a(i) = \frac{1}{|C_i| - 1} \sum_{j \in C_i, j \neq i} d(i, j)$$

where C_i is the cluster to which i belongs and $d(i, j)$ is the distance between points i and j.

5.3 Separation Term $b(i)$

$b(i)$ is:

$$b(i) = \min_{C_k \neq C_i} \left(\frac{1}{|C_k|} \sum_{j \in C_k} d(i, j) \right)$$

5.4 Interpretation of Silhouette Score

The Silhouette Score $S(i)$ for point i:

- $S(i) \approx 1$: Point is well-matched to its own cluster.
- $S(i) \approx 0$: Point lies close to the boundary.
- $S(i) \approx -1$: Point likely assigned to the wrong cluster.

6 Internal Clustering Measures

These measures assess clustering quality without true labels.

6.1 Dunn Index

The Dunn Index assesses the ratio of minimum inter-cluster distance to maximum intra-cluster distance:

$$D = \frac{\min_{i \neq j} \delta(C_i, C_j)}{\max_k \Delta(C_k)}$$

where:

- $\delta(C_i, C_j)$ is the distance between clusters C_i and C_j.
- $\Delta(C_k)$ is the diameter of cluster C_k.

6.2 Davies-Bouldin Index

The Davies-Bouldin Index quantifies similarity between each cluster and its most similar cluster:

$$DB = \frac{1}{K} \sum_{i=1}^{K} \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(C_i, C_j)} \right)$$

where σ_i and σ_j are average distances from points in clusters C_i and C_j to their respective centroids, and $d(C_i, C_j)$ is the distance between centroids.

7 Adjusted Rand Index (ARI)

ARI evaluates clustering similarity, adjusted for chance.

7.1 Rand Index (RI)

The Rand Index measures the agreement between two clusterings by considering all pairs of points and how they are grouped. For n data points, we consider all $\binom{n}{2}$ pairs of points.

7.1.1 Four Cases for Point Pair Comparison

For each pair of points (i, j), we have four possible outcomes:

- **True Positive (TP)**: Points are in the same cluster in **both** clusterings
- **True Negative (TN)**: Points are in different clusters in **both** clusterings
- **False Positive (FP)**: Points are in the same cluster in clustering A but different clusters in clustering B
- **False Negative (FN)**: Points are in different clusters in clustering A but same cluster in clustering B

7.1.2 Contingency Table Representation

Let $U = \{U_1, U_2, ..., U_R\}$ and $V = \{V_1, V_2, ..., V_C\}$ be two clusterings. We can create a contingency table:

	V_1	V_2	$\cdots$	V_C	Total
U_1	n_{11}	n_{12}	$\cdots$	n_{1C}	a_1
U_2	n_{21}	n_{22}	$\cdots$	n_{2C}	a_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
U_R	n_{R1}	n_{R2}	$\cdots$	n_{RC}	a_R
Total	b_1	b_2	$\cdots$	b_C	n

Where:

- n_{ij} = number of points in cluster U_i from first clustering and cluster V_j from second clustering
- $a_i = \sum_{j=1}^{C} n_{ij}$ = total points in cluster U_i
- $b_j = \sum_{i=1}^{R} n_{ij}$ = total points in cluster V_j

7.1.3 Computing Pair Counts from Contingency Table

The counts of pairs can be computed as:

$$\text{TP} = \sum_{i=1}^{R}\sum_{j=1}^{C}\binom{n_{ij}}{2} \quad \text{(Pairs agreeing on same cluster)}$$

$$\text{FP} = \sum_{i=1}^{R}\binom{a_i}{2} - \sum_{i=1}^{R}\sum_{j=1}^{C}\binom{n_{ij}}{2}$$

$$\text{FN} = \sum_{j=1}^{C}\binom{b_j}{2} - \sum_{i=1}^{R}\sum_{j=1}^{C}\binom{n_{ij}}{2}$$

$$\begin{aligned}\text{TN} &= \binom{n}{2} - \text{TP} - \text{FP} - \text{FN} \\ &= \binom{n}{2} - \sum_{i=1}^{R}\binom{a_i}{2} - \sum_{j=1}^{C}\binom{b_j}{2} + \sum_{i=1}^{R}\sum_{j=1}^{C}\binom{n_{ij}}{2}\end{aligned}$$

7.1.4 Rand Index Formula

$$\text{RI} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} = \frac{\text{TP} + \text{TN}}{\binom{n}{2}}$$

7.2 Adjusted Rand Index (ARI)

The ARI adjusts the Rand Index for chance agreement, providing a more reliable measure.

7.2.1 Motivation for Adjustment

The raw RI has limitations:

- RI tends to increase with the number of clusters
- Random clusterings can achieve high RI values by chance
- RI range is [0,1] but expected value is not 0 for random clustering

7.2.2 Mathematical Definition

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{1 - \mathbb{E}[\text{RI}]}$$

7.2.3 Computing Expected Rand Index

Under the hypergeometric model of randomness, where cluster sizes are fixed but assignments are random:

$$\mathbb{E}[\text{RI}] = \frac{\sum_{i=1}^{R} \binom{a_i}{2} \sum_{j=1}^{C} \binom{b_j}{2}}{\binom{n}{2}^2} + \frac{\binom{n}{2} - \sum_{i=1}^{R} \binom{a_i}{2} - \sum_{j=1}^{C} \binom{b_j}{2} + \sum_{i=1}^{R} \sum_{j=1}^{C} \binom{n_{ij}}{2}}{\binom{n}{2}}$$

More compactly, the expected index can be computed as:

$$\mathbb{E}[\text{Index}] = \frac{\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}{\binom{n}{2}^2}$$

7.2.4 Practical Computation Formula

The ARI can be computed directly from the contingency table:

$$\text{ARI} = \frac{\sum_{ij} \binom{n_{ij}}{2} - \frac{\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}{\binom{n}{2}}}{\frac{1}{2}\left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right] - \frac{\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}{\binom{n}{2}}}$$

7.3 Detailed Example

7.3.1 Example Data

Consider two clusterings of 6 points:

- **Clustering A**: $\{1, 2, 3\}, \{4, 5, 6\}$
- **Clustering B**: $\{1, 2\}, \{3, 4, 5, 6\}$

Contingency table:

	$\{1, 2\}$	$\{3, 4, 5, 6\}$	Total
$\{1, 2, 3\}$	2	1	3
$\{4, 5, 6\}$	0	3	3
Total	2	4	6

7.3.2 Step 1: Compute Pair Counts

$$\text{TP} = \binom{2}{2} + \binom{1}{2} + \binom{0}{2} + \binom{3}{2} = 1 + 0 + 0 + 3 = 4$$

$$\sum_i \binom{a_i}{2} = \binom{3}{2} + \binom{3}{2} = 3 + 3 = 6$$

$$\sum_j \binom{b_j}{2} = \binom{2}{2} + \binom{4}{2} = 1 + 6 = 7$$

$$\text{FP} = 6 - 4 = 2$$

$$\text{FN} = 7 - 4 = 3$$

$$\text{TN} = \binom{6}{2} - 6 - 7 + 4 = 15 - 6 - 7 + 4 = 6$$

7.3.3 Step 2: Compute Rand Index

$$\text{RI} = \frac{4 + 6}{4 + 6 + 2 + 3} = \frac{10}{15} = 0.667$$

7.3.4 Step 3: Compute Expected Rand Index

$$\mathbb{E}[\text{RI}] = \frac{6 \times 7}{15^2} = \frac{42}{225} = 0.187$$

7.3.5 Step 4: Compute Adjusted Rand Index

$$\text{ARI} = \frac{0.667 - 0.187}{1 - 0.187} = \frac{0.480}{0.813} = 0.590$$

7.4 Interpretation of ARI Values

- **ARI = 1**: Perfect agreement between clusterings
- **ARI = 0**: Agreement equivalent to random chance
- **ARI ¡ 0**: Less agreement than expected by chance
- **ARI ¿ 0.9**: Excellent agreement
- **ARI ¿ 0.8**: Strong agreement
- **ARI ¿ 0.65**: Moderate agreement
- **ARI ¡ 0.5**: Weak agreement

7.5 Practical Implementation

7.5.1 Python Implementation

```
from sklearn.metrics import adjusted_rand_score
import numpy as np
from math import factorial

def comb(n, k):
    return factorial(n) // (factorial(k) * factorial(n-k))

def manual_ari(labels_true, labels_pred):
    n = len(labels_true)
    contingency = {}

    # Build contingency table
    for i in range(n):
        key = (labels_true[i], labels_pred[i])
        contingency[key] = contingency.get(key, 0) + 1

    # Compute sums
    a_sum = sum(comb(n_i, 2) for n_i in
                np.unique(labels_true, return_counts=True)[1])
    b_sum = sum(comb(n_j, 2) for n_j in
                np.unique(labels_pred, return_counts=True)[1])

    index = sum(comb(n_ij, 2) for n_ij in contingency.values())
    expected_index = a_sum * b_sum / comb(n, 2)
    max_index = 0.5 * (a_sum + b_sum)

    return (index - expected_index) / (max_index - expected_index)

# Example usage
labels_true = [0, 0, 0, 1, 1, 1]
labels_pred = [0, 0, 1, 1, 1, 1]
print(f"Manual ARI: {manual_ari(labels_true, labels_pred):.3f}")
print(f"Sklearn ARI: {adjusted_rand_score(labels_true, labels_pred):.3f}")
```

7.6 Advantages of ARI over RI

- **Chance Correction**: Accounts for agreement expected by random chance
- **Comparability**: Allows meaningful comparison across different datasets
- **Interpretability**: 0 represents random labeling, 1 represents perfect agreement
- **Symmetry**: ARI(A,B) = ARI(B,A)

7.7 Limitations and Considerations

- Assumes clusterings are independent under the null model
- May be sensitive to the number of clusters and cluster sizes
- Requires knowledge of true labels (supervised measure)
- Computational complexity $O(n^2)$ for large datasets

8 Cross-Validation in Clustering

Without true labels, cross-validation (CV) in clustering is used to assess the *stability* and *generalizability* of the clustering result. A common k-fold CV procedure is:

1. **Split:** Randomly partition the dataset into k folds.
2. **Train and Assign:** For each fold i:
   (a) Use the other $k-1$ folds as the training set. Perform clustering (e.g., K-Means) on this training set to obtain K centroids.
   (b) Assign each point in the held-out fold i to the nearest centroid from the training set.
3. **Evaluate:** After processing all folds, every point in the dataset has been assigned a cluster label. An internal validation measure, such as the **Silhouette Score**, can now be computed on the *entire dataset* using these assigned labels. A high score indicates a stable clustering structure that generalizes across different data subsets.

8.1 Stability Assessment

The stability of clustering can be assessed by:

- Comparing cluster assignments across different folds
- Measuring the consistency of cluster centroids
- Computing the variance of internal validation scores across folds

9 Application: Clustering in Image Segmentation

Clustering can be applied to group pixels with similar colors in image segmentation, using K-Means clustering in RGB color space to group pixels into regions based on color similarity.

10 Spectral Clustering Methodology

Spectral clustering is a technique that leverages the eigenvalues and eigenvectors of a data similarity matrix to project data into a lower-dimensional space for clustering. This methodology is particularly useful for identifying clusters when data points are not well-separated in the original space but lie on a low-dimensional manifold.

The general steps for spectral clustering are as follows:

1. **Construct a similarity matrix S:** For a set of n data points, form an $n \times n$ similarity matrix S where each element S_{ij} represents the similarity (e.g., Gaussian kernel similarity) between points i and j.

2. **Form the Laplacian matrix L:** The Laplacian matrix L can be constructed in various forms:

   - Unnormalized Laplacian: $L = D - S$
   - Normalized Symmetric Laplacian: $L_{sym} = I - D^{-1/2}SD^{-1/2}$
   - Random Walk Laplacian: $L_{rw} = I - D^{-1}S$

   where D is the degree matrix, a diagonal matrix where $D_{ii} = \sum_j S_{ij}$.

3. **Compute the smallest k eigenvectors of L:** Let k be the number of clusters desired. Compute the first k smallest eigenvalues and their corresponding eigenvectors of L.

4. **Form the matrix E:** Construct an $n \times k$ matrix E, where each row corresponds to a data point and each column corresponds to one of the k smallest eigenvectors. Each row represents a transformed k-dimensional representation of the original data point.

5. **Cluster the rows of E:** Apply k-means clustering on the rows of E. Each row in E is now a point in k-dimensional space, and k-means will assign each point to one of k clusters.

6. **Assign cluster labels:** Finally, assign each data point in the original space to its corresponding cluster based on the k-means results.

11 Numerical Example

Consider five data points in a 2D space:

$$\mathbf{X} = \{(0, 1), (1, 0), (0, -1), (-1, 0), (2, 2)\}$$

We want to separate these points into two clusters.

11.1 Step 1: Construct Similarity Matrix

Construct a similarity matrix S using Gaussian similarity:

$$S_{ij} = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$$

For simplicity, let's choose $\sigma = 1$ and compute S. For example:

$$S_{12} = \exp\left(-\frac{\|(0,1) - (1,0)\|^2}{2}\right) = \exp\left(-\frac{(\sqrt{2})^2}{2}\right) = \exp(-1) \approx 0.368$$

$$S_{15} = \exp\left(-\frac{\|(0,1) - (2,2)\|^2}{2}\right) = \exp\left(-\frac{(\sqrt{5})^2}{2}\right) = \exp(-2.5) \approx 0.082$$

The complete similarity matrix:

$$S = \begin{bmatrix} 1.000 & 0.368 & 0.135 & 0.368 & 0.082 \\ 0.368 & 1.000 & 0.368 & 0.135 & 0.105 \\ 0.135 & 0.368 & 1.000 & 0.368 & 0.105 \\ 0.368 & 0.135 & 0.368 & 1.000 & 0.082 \\ 0.082 & 0.105 & 0.105 & 0.082 & 1.000 \end{bmatrix}$$

11.2 Step 2: Form the Laplacian Matrix

Calculate the degree matrix D:

$$D = \begin{bmatrix} 1.953 & 0 & 0 & 0 & 0 \\ 0 & 1.976 & 0 & 0 & 0 \\ 0 & 0 & 1.976 & 0 & 0 \\ 0 & 0 & 0 & 1.953 & 0 \\ 0 & 0 & 0 & 0 & 1.374 \end{bmatrix}$$

The unnormalized Laplacian L is:

$$L = D - S = \begin{bmatrix} 0.953 & -0.368 & -0.135 & -0.368 & -0.082 \\ -0.368 & 0.976 & -0.368 & -0.135 & -0.105 \\ -0.135 & -0.368 & 0.976 & -0.368 & -0.105 \\ -0.368 & -0.135 & -0.368 & 0.953 & -0.082 \\ -0.082 & -0.105 & -0.105 & -0.082 & 0.374 \end{bmatrix}$$

11.3 Step 3: Compute Eigenvectors

Calculate the smallest two eigenvalues and their corresponding eigenvectors of L. Suppose the resulting eigenvectors are:

$$\mathbf{v}_1 = [0.447, 0.447, 0.447, 0.447, -0.894]^T, \quad \mathbf{v}_2 = [0.500, -0.500, 0.500, -0.500, 0.000]^T$$

11.4 Step 4: Form Matrix E

Construct matrix E using $\mathbf{v}_1$ and $\mathbf{v}_2$:

$$E = \begin{bmatrix} 0.447 & 0.500 \\ 0.447 & -0.500 \\ 0.447 & 0.500 \\ 0.447 & -0.500 \\ -0.894 & 0.000 \end{bmatrix}$$

11.5 Step 5: Apply K-Means Clustering

Run k-means on the rows of E with $k = 2$, resulting in two clusters:

$$\text{Cluster 1: } \{(0,1),(0,-1),(2,2)\} \quad \text{Cluster 2: } \{(1,0),(-1,0)\}$$

11.6 Conclusion

Spectral clustering allows for effective clustering by leveraging the eigenstructure of the similarity graph. This method excels in scenarios where data points are connected in non-linear manifolds or complex structures, as shown in this example.

12 Comparison of Clustering Methods

Method	**Strengths**	**Limitations**
K-Means	- Computationally efficient - Scalable to large datasets - Simple to implement	- Assumes spherical clusters - Sensitive to initial centroids - Requires specifying K
Spectral Clustering	- Handles non-convex clusters - Works on complex manifolds - Robust to cluster shape	- Computationally expensive - Memory intensive for large n - Sensitive to similarity measure
Hierarchical Clustering	- No need to specify K - Provides dendrogram visualization - Flexible cluster shapes	- High computational complexity - Sensitive to noise and outliers - Difficult to handle large datasets

13 Conclusion

This document has provided a comprehensive overview of clustering techniques, evaluation metrics, and applications. Key clustering methods include:

- **K-Means**: Efficient for spherical clusters, benefits from K-Means++ initialization and Elkan's acceleration
- **Spectral Clustering**: Powerful for non-convex clusters and complex data manifolds
- **Hierarchical Clustering**: Provides multi-resolution clustering through dendrograms

Evaluation measures can be categorized as:

- **Internal measures** (Silhouette Score, Dunn Index, Davies-Bouldin): Assess quality without ground truth
- **External measures** (Adjusted Rand Index): Compare against known labels
- **Stability assessment** (Cross-Validation): Test generalizability across data subsets

Clustering finds applications in diverse domains including image segmentation, customer segmentation, document clustering, and anomaly detection. The choice of clustering method depends on the data characteristics, computational constraints, and the specific requirements of the application.

A Detailed Elkan's Method Example

A.1 Problem Setup

Given 12 points with integer coordinates:

$$\begin{array}{lll} P1:(1,1) & P5:(3,3) & P9:(7,1) \\ P2:(1,4) & P6:(4,2) & P10:(7,4) \\ P3:(2,2) & P7:(5,1) & P11:(8,2) \\ P4:(2,5) & P8:(5,3) & P12:(8,5) \end{array}$$

Initial centroids: $C1 = (1,1)$, $C2 = (4,2)$, $C3 = (7,4)$

A.2 Step 1: Initial Assignment

Calculate all distances and assign points:

- Cluster 1: P1, P2, P3 (closest to C1)

- Cluster 2: P4, P5, P6, P7, P8 (closest to C2)
- Cluster 3: P9, P10, P11, P12 (closest to C3)

Update centroids:

$$C1_{new} = (1.33, 2.33) \quad \text{Movement: } 1.53$$
$$C2_{new} = (3.8, 2.6) \quad \text{Movement: } 0.63$$
$$C3_{new} = (7.5, 3) \quad \text{Movement: } 1.12$$

A.3 Step 2: Bounds-Based Assignment

Update bounds using centroid movements:

- Upper bounds: $u_{new}(i) = u_{old}(i) + \text{movement of assigned centroid}$
- Lower bounds: $l_{new}(i, j) = \max(0, l_{old}(i, j) - \text{movement of centroid } j)$

For P3 (2,2) assigned to C1:

$$u(P3) = 1.41 + 1.53 = 2.94$$
$$l(P3, C2) = \max(0, 2.0 - 0.63) = 1.37$$
$$l(P3, C3) = \max(0, 5.39 - 1.12) = 4.27$$

Since $u(P3) > l(P3, C2)$, recalculate distance to C2. New distance to C2 is 1.84, so P3 remains with C1.

A.4 Step 3: Convergence

No centroid movement in Step 2, so bounds remain unchanged and no distance calculations needed. Algorithm converges.

A.5 Optimization Benefits

- Step 1: 36 distance calculations (no optimization)
- Step 2: $\sim$20 distance calculations (44% reduction)
- Step 3: 0 distance calculations (100% reduction)
- Total savings: $\sim$42% of distance calculations

Elkan's method significantly accelerates K-Means clustering by leveraging triangle inequality and maintaining bounds, reducing computational cost while guaranteeing identical results to standard K-Means. The method is particularly beneficial for large datasets and high-dimensional spaces.